

Fan Director Studio — Developer Handoff Guide

Project Overview

Fan Director Studio is a fictional demonstration of a boutique creator commission platform. Maya Atelier (demo creator) uses this to collaboratively plan custom commissions with fans through an AI Director interface, followed by creator approval workflows.

Important: This is a design prototype, not production software. All AI interactions are mocked, no real payments are processed, and all content is for demonstration purposes only.

Tech Stack Recommendations

Frontend

- **Framework:** React 18+ or Vue 3 (component-based architecture)
- **Styling:** Tailwind CSS (PlayCDN version for prototyping)
- **Icons:** Iconify (lucide icon set)
- **Fonts:** Google Fonts (Cormorant Garamond, Inter, JetBrains Mono)
- **State Management:** React Context or Pinia (Vue)
- **Routing:** React Router or Vue Router (full page navigation)

Backend (Future)

- **API Framework:** Express.js, Django, or FastAPI
 - **Database:** PostgreSQL for orders, users, creator boundaries
 - **Payment Processing:** Stripe, Square (NOT implemented in prototype)
 - **File Storage:** S3 or similar for video uploads
 - **Real AI:** OpenAI API (if implementing actual Director chat)
-

Installation & Setup

Dependencies

```
# Frontend
npm install react-router-dom axios zustand
npm install -D tailwindcss postcss autoprefixer
```

```
npm install iconify-icon
```

```
# Fonts
```

```
# Add to index.html <head>:
```

```
<link href="https://fonts.googleapis.com/css2?family=Cormorant+Garamond:wght@400;500;600;700&family=Inter:wght@300;400;500;600;700;800;900" rel="stylesheet">
```

```
# Icons
```

```
<script src="https://code.iconify.design/iconify-icon/1.0.7/iconify-icon.min.js"></script>
```

Tailwind Configuration

```
// tailwind.config.js
export default {
  theme: {
    extend: {
      fontFamily: {
        serif: ['Cormorant Garamond', 'serif'],
        sans: ['Inter', 'sans-serif'],
        mono: ['JetBrains Mono', 'monospace'],
      },
      colors: {
        cream: '#FDF8F3',
        panel: '#FFFFFF',
        secondary: '#F5F0EB',
        espresso: '#302720',
        muted: '#70625C',
        rose: '#E4A4BD',
        deepprose: '#84485D',
        divider: '#DED3CB',
        status: {
          new: '#4B9FE3',
          pending: '#D4A574',
          alert: '#E37A6A',
          approved: '#8EB486'
        }
      },
      transitionDuration: {
        '160': '160ms',
      },
      maxWidth: {
        container: '1320px'
      },
      spacing: {
        'container-x': '32px'
      }
    }
  }
}
```

File Structure

```
src/
├── components/
│   ├── Header.jsx           # Sticky navigation (all pages)
│   ├── RequestCard.jsx      # Creator dashboard card
│   ├── Button.jsx           # Primary/Secondary buttons
│   ├── SceneCard.jsx        # Live preview panel
│   ├── ProgressTrail.jsx    # 4-step journey
│   ├── Modal.jsx            # Modal wrapper (backdrop + close)
│   ├── StatusBadge.jsx      # Status indicators
│   ├── Timeline.jsx         # Next steps timeline
│   └── Input.jsx            # Form inputs
├── pages/
│   ├── BoutiqueEntrance.jsx # Theme selection
│   ├── AIDirector.jsx       # Conversation + live card
│   ├── ReviewScene.jsx      # Final review before sending
│   ├── SendConfirmation.jsx  # Success screen
│   ├── CreatorDashboard.jsx  # Request queue
│   ├── CreatorDetailModal.jsx # Request details + actions
│   ├── AskQuestionModal.jsx  # Question form
│   └── DeclineConfirmationModal.jsx # Decline confirmation
├── hooks/
│   ├── useRequest.js         # Fetch request data
│   ├── useSceneCard.js       # Scene card state
│   └── useCreatorDashboard.js # Dashboard state
├── context/
│   ├── AuthContext.jsx       # User authentication
│   ├── RequestContext.jsx    # Commission request data
│   └── UIContext.jsx         # Modal/UI state
├── styles/
│   ├── globals.css           # Global styles + animations
│   ├── components.css        # Component-specific styles
│   └── responsive.css        # Responsive overrides
├── utils/
│   ├── api.js                # API client
│   ├── colors.js             # Color constants
│   └── helpers.js             # Utility functions
├── types/
│   ├── request.ts            # Request interface
│   ├── user.ts               # User interface
│   └── scene.ts               # Scene interface
├── App.jsx                   # Router configuration
└── main.jsx                   # Entry point
```

Component Implementation Guide

1. Header Navigation

File: src/components/Header.jsx**Key Props:**

- `currentPage` : string (highlights active nav)
- `links` : array of nav links

Implementation Notes:

- Sticky positioning (top-0, z-50)
- Backdrop blur effect
- Responsive: nav hidden on mobile (use hamburger menu)
- Logo links to home
- All nav links use React Router

Example:

```
export function Header({ currentPage }) {
  return (
    <header className="sticky top-0 z-50 bg-cream/95 backdrop-blur-sm border-b border-divider">
      <div className="max-w-container mx-auto px-4 sm:px-6 lg:px-8 h-[80px] flex items-center justify-between">
        <Link to="/" className="font-serif text-2xl sm:text-3xl font-medium tracking-tight">
          Maya Atelier
        </Link>
        <span className="inline-flex items-center px-2 py-0.5 rounded text-xs font-medium bg-secondary text-muted bo
          Demo
        </span>
        <nav className="hidden md:flex items-center gap-8">
          <NavLink to="/#collection" className="text-sm font-medium hover:text-muted transition-colors">Collection</
          <NavLink to="/#studio" className="text-sm font-medium hover:text-muted transition-colors">Your studio</Nav
          <NavLink to="/#saved" className="text-sm font-medium hover:text-muted transition-colors">Saved ideas</NavL
        </nav>
      </div>
    </header>
  )
}
```

2. Request Card (Creator Dashboard)**File:** src/components/RequestCard.jsx**Key Props:**

- `request` : object (fan handle, title, price, delivery, status)
- `onClick` : function (navigate to detail modal)

Implementation Notes:

- Entire card is clickable (use or )
- Status icon changes based on status type
- Hover: shadow lift + deeprse text color
- Avatar: use initials if no image
- Line clamp description (2 lines max)

Example:

```
export function RequestCard({ request, onClick }) {
  const statusConfig = {
    new: { icon: 'lucide:circle-dot', color: 'text-status-new', label: 'New' },
    pending: { icon: 'lucide:clock', color: 'text-status-pending', label: 'Awaiting Reply' },
    alert: { icon: 'lucide:alert-triangle', color: 'text-status-alert', label: 'Changes Pending' },
    approved: { icon: 'lucide:check-circle', color: 'text-status-approved', label: 'Approved' }
  }

  const status = statusConfig[request.status]

  return (
    <button
      onClick={onClick}
      className="block w-full bg-panel border border-divider rounded-[12px] p-6 shadow-sm hover:shadow-md transition
    >
      <div className="flex justify-between items-start mb-4">
        <div className="flex items-center gap-3">
          <div className="w-8 h-8 rounded-full bg-cream border border-divider flex items-center justify-center text-
            {request.fanHandle.slice(0, 2).toUpperCase()}
          </div>
          <span className="text-sm font-medium text-muted">@{request.fanHandle}</span>
        </div>
        <div className="flex items-center gap-2">
          <span className={`text-xs font-medium ${status.color}`}>{status.label}</span>
          <iconify-icon icon={status.icon} className={status.color} width="18"></iconify-icon>
        </div>
      </div>

      <h3 className="font-serif text-2xl font-bold mb-2 group-hover:text-deeprse transition-colors duration-160">
        {request.title}
      </h3>
      <p className="text-sm text-muted mb-6 line-clamp-2">
        {request.description}
      </p>

      <hr className="border-t border-divider mb-4" />

      <div className="flex items-center justify-between">
        <div className="flex flex-col">
          <span className="text-xs text-muted mb-1">Estimated Total</span>
          <span className="text-base font-semibold">${request.estimatedPrice}</span>
        </div>
      </div>
    </button>
  )
}
```

```

    </div>
    <div className="flex flex-col text-right">
      <span className="text-xs text-muted mb-1">Requested Delivery</span>
      <span className="text-sm font-medium">{request.deliveryDate}</span>
    </div>
  </div>
</button>
)
}

```

3. Primary Button

File: `src/components/Button.jsx`

Key Props:

- `variant`: "primary" | "secondary"
- `size`: "sm" | "md" | "lg"
- `icon`: optional icon name
- `disabled`: boolean
- `onClick`: function

Implementation Notes:

- Primary: rose background, espresso text, shadow
- Secondary: transparent bg, divider border, white on hover
- All variants: 48px min height
- Focus ring: 2px espresso outline with 2px offset

Example:

```

export function Button({ variant = 'primary', size = 'md', icon, children, ...props }) {
  const baseClasses = "rounded-[12px] font-medium transition-colors focus-ring flex items-center justify-center gap-2";

  const variants = {
    primary: "bg-rose text-espresso hover:bg-[#D595AE] shadow-sm",
    secondary: "border border-divider bg-transparent text-espresso hover:bg-white"
  };

  const sizes = {
    sm: "px-6 h-[44px] text-sm",
    md: "px-8 h-[48px] text-base",
    lg: "px-10 h-[52px] text-lg"
  };

  return (

```

```

<button className={` ${baseClasses} ${variants[variant]} ${sizes[size]} `} {...props}>
  {children}
  {icon && <iconify-icon icon={icon} width="18"></iconify-icon>}
</button>
)
}

```

4. Scene Card Panel

File: `src/components/SceneCard.jsx`

Key Props:

- `scene`: object (title, description, components, pricing, timeline)
- `isSticky`: boolean (sticky on desktop)
- `editable`: boolean (show edit links)

Implementation Notes:

- Multiple sections with divider borders
- Responsive padding (6px mobile, 10px desktop)
- Components grid: 1 col mobile, 3 cols desktop
- Price breakdown: line items with edit icons
- Budget indicator: green if under budget, red if over
- Timeline: 7 days post-payment

Example:

```

export function SceneCard({ scene, isSticky = true, editable = false }) {
  const budgetStatus = scene.estimatedTotal <= scene.budget

  return (
    <div className={isSticky ? "sticky top-[104px]" : ""}>
      <div className="bg-panel rounded-[12px] border border-divider shadow-[0_8px_30px_rgba(0,0,0,0.04)] overflow-hidden">

        {/* Header */}
        <div className="p-6 sm:p-10 border-b border-divider">
          <h2 className="font-serif text-3xl font-semibold mb-3">{scene.title}</h2>
          <p className="text-muted text-sm">{scene.description}</p>
        </div>

        {/* Components Grid */}
        <div className="p-6 sm:p-10 border-b border-divider">
          <h3 className="text-xs font-semibold uppercase tracking-wider text-muted mb-6">Included Components</h3>
          <div className="grid grid-cols-1 sm:grid-cols-3 gap-4">
            {scene.components.map(comp => (

```

```

    <div key={comp.id} className="p-5 rounded-xl border border-divider">
      <div className="flex items-center gap-2 mb-2">
        <iconify-icon icon={comp.icon} className="text-rose text-lg"></iconify-icon>
        <span className="font-medium text-sm text-espresso">{comp.name}</span>
      </div>
      <p className="text-sm text-muted">{comp.value}</p>
    </div>
  )}}
</div>
</div>

{/* Price Breakdown */}
<div className="p-6 sm:p-10 bg-secondary">
  <h3 className="text-xs font-semibold uppercase tracking-wider text-muted mb-5">Estimated Price</h3>
  <div className="space-y-3 mb-5">
    {scene.lineItems.map(item => (
      <div key={item.id} className="flex justify-between items-center text-sm">
        <span className="text-muted">{item.label}</span>
        <span className="text-espresso font-medium">${item.price}</span>
      </div>
    ))}
  </div>
  <div className="pt-4 border-t border-divider flex justify-between items-end mb-4">
    <span className="font-medium text-espresso">Total Estimate</span>
    <span className="font-serif font-bold text-3xl text-espresso">${scene.estimatedTotal}</span>
  </div>

  {/* Budget Indicator */}
  <div className={`flex items-center gap-2 text-xs font-medium px-3 py-2 rounded-lg border w-fit ${
    budgetStatus
      ? 'bg-green-50 text-green-800 border-green-200'
      : 'bg-red-50 text-red-800 border-red-200'
    }`} >
    <iconify-icon icon={budgetStatus ? 'lucide:check-circle-2' : 'lucide:alert-circle'}></iconify-icon>
    {budgetStatus
      ? `${scene.budget - scene.estimatedTotal} remaining of ${scene.budget} budget`
      : `${scene.estimatedTotal - scene.budget} over budget`}
  </div>
</div>
</div>
</div>
)
}

```

5. Progress Trail / Stepper

File: src/components/ProgressTrail.jsx

Key Props:

- `steps` : array of step objects
- `currentStep` : number (0-indexed)

Implementation Notes:

- Active step: rose background (rose/20) + espresso text + bold
- Future steps: muted text
- Chevron separators
- Responsive: can collapse to step numbers on mobile

Example:

```
export function ProgressTrail({ steps, currentStep }) {
  return (
    <div className="flex items-center gap-2 text-sm text-muted flex-wrap sm:flex-nowrap">
      {steps.map((step, idx) => (
        <div key={idx} className="flex items-center gap-2">
          <span className={idx === currentStep ? "font-medium text-espresso bg-rose/20 px-2 py-1 rounded-md" : ""}>
            {step.label}
          </span>
          {idx < steps.length - 1 && <iconify-icon icon="lucide:chevron-right" className="text-xs hidden sm:inline">
            </div>
          </div>
        ))}
      </div>
    )
  }
}
```

6. Modal Wrapper

File: `src/components/Modal.jsx`

Key Props:

- `isOpen` : boolean
- `onClose` : function
- `title` : string
- `size` : "sm" | "md" | "lg"
- `children` : ReactNode

Implementation Notes:

- Fixed backdrop: inset-0 with bg-black/30
- Z-index: 50 (above all content)
- Close button: X top-right, links back to dashboard

- Content area: flex row for desktop, column for mobile
- Scrollable body: max-h-full overflow-y-auto

Example:

```
export function Modal({ isOpen, onClose, title, size = 'md', children }) {
  if (!isOpen) return null

  const maxWidth = { sm: 'max-w-[480px]', md: 'max-w-[800px]', lg: 'max-w-[1100px]' }

  return (
    <div className="fixed inset-0 bg-black/30 flex items-center justify-center z-50">
      <div className={`bg-panel rounded-[12px] shadow-[0_20px_60px_rgba(0,0,0,0.08)] w-full ${maxWidth[size]} flex f

        {/* Close Button */}
        <button
          onClick={onClose}
          className="absolute top-4 right-4 z-10 w-10 h-10 flex items-center justify-center rounded-full bg-cream/80
        >
          <iconify-icon icon="lucide:x" width="24"></iconify-icon>
        </button>

        {/* Body */}
        <div className="flex-1 overflow-y-auto p-8">
          {title && <h2 className="font-serif text-3xl font-semibold mb-6">{title}</h2>}
          {children}
        </div>
      </div>
    </div>
  )
}
```

Page-Specific Implementation

Boutique Entrance

Route: /

State:

- `selectedTheme` : selected theme object
- `customIdea` : custom idea text

Components Used:

- Header

- Hero section with image
- 3 theme cards (clickable)
- Custom idea form

Key Interactions:

- Click theme card → navigate to `/ai-director`
 - Submit custom idea → navigate to `/ai-director` with custom idea
-

AI Director Workspace

Route: `/ai-director`

State:

- `conversation`: array of message objects
- `sceneCard`: live scene card data
- `userInput`: composer input

Components Used:

- Header
- ProgressTrail
- Conversation thread (7-column)
- SceneCard (5-column, sticky desktop)
- Composer input with send/suggest/undo

Key Interactions:

- Type message → send → append to conversation + update scene card
- Click choice card → update scene + adjust price
- Click "Review Scene Card" → navigate to `/review`

API Calls:

- `POST /api/director/message` (send fan message)
 - `POST /api/director/choice` (select a choice)
 - `GET /api/scene/:id` (fetch live scene card)
-

Review Scene Card

Route: `/review`

State:

- `scene` : complete scene card data

Components Used:

- Header
- ProgressTrail
- Full SceneCard with image + components + pricing
- Creator Boundaries section (collapsible)
- Back & Send CTAs

Key Interactions:

- "Back to Edit" → navigate back to `/ai-director`
 - "Send to Creator" → `POST /api/requests` + navigate to `/confirmation`
-

Send Confirmation

Route: `/confirmation`

State:

- `order` : order details object

Components Used:

- Header
- Success icon with bounce-in animation
- Order details card
- Timeline (4 steps, only step 1 active)
- Save idea + Back CTAs

Key Interactions:

- "Save Idea" → local state (no API needed)
 - "Back to Collection" → navigate to `/`
-

Creator Dashboard

Route: `/creator/requests`

State:

- `requests` : array of request objects
- `filter` : status filter
- `sort` : sort order

Components Used:

- Header with nav tabs (Requests active)
- Filter/sort dropdowns
- RequestCard (2-column grid)
- 6 mock request cards

Key Interactions:

- Filter by status → re-render cards
- Sort by date/price → re-render cards
- Click request card → open detail modal

API Calls:

- `GET /api/creator/requests?status=&sort=` (fetch requests)
-

Creator Detail Modal

Route: Overlay on `/creator/requests`

State:

- `selectedRequest` : request object
- `modalState` : ask/decline modals

Components Used:

- Modal wrapper
- Scene card (left, read-only)
- Decision controls (right: Approve, Propose, Ask, Decline)

Key Interactions:

- "Ask Question" → open Ask Question Modal
 - "Decline" → open Decline Confirmation Modal
 - "Approve" or "Propose Changes" → local state (demo only)
 - Close (X) → close modal
-

Ask Question Modal

Route: Nested modal inside Creator Detail Modal

State:

- `formData` : { name, email, message }

Components Used:

- Modal wrapper
- 3 inputs (name, email, message textarea)
- Send Message & Cancel buttons

Key Interactions:

- "Send Message" → `POST /api/creator/ask` + close modal
 - "Cancel" → close modal without action
 - Close (X) → close modal
-

Decline Confirmation Modal

Route: Nested modal inside Creator Detail Modal

State: None (purely presentational)

Components Used:

- Modal wrapper
- Alert icon + message
- Cancel & Confirm Decline buttons

Key Interactions:

- "Cancel" → close modal
 - "Confirm Decline" → `POST /api/creator/decline` + close modal + remove from requests list
 - Close (X) → close modal
-

Data Models

Request Object

```

interface Request {
  id: string;           // UUID
  fanHandle: string;    // @username
  title: string;        // Theme title
  description: string;  // Short summary
  scene: SceneCard;     // Full scene card data
  estimatedPrice: number; // $xxx.xx
  budget: number;       // Fan's budget
  deliveryDate: string; // "Dec 12"
  status: 'new' | 'pending' | 'alert' | 'approved';
  createdAt: ISO8601;
  updatedAt: ISO8601;
}

interface SceneCard {
  id: string;
  title: string;           // "Vintage Lounge Greeting"
  description: string;
  components: Component[];
  lineItems: LineItem[];
  estimatedTotal: number;
  timeline: Timeline;
  referenceImage: string; // URL
  wardrobe: string;       // Creator's choice note
}

interface Component {
  id: string;
  name: string;           // "Duration"
  value: string;          // "3-Minute Video"
  icon: string;           // Lucide icon name
  price: number;
}

interface LineItem {
  id: string;
  label: string;          // "Base Video (3-min)"
  price: number;
  editable: boolean;
}

interface Timeline {
  estimatedDelivery: string; // "7 days post-payment"
  creatorReviewTime: string; // "24-48 hours"
}

interface User {
  id: string;
  email: string;
  role: 'fan' | 'creator' | 'admin';
  creatorHandle: string; // if creator
  profile: {
    name: string;
  }
}

```

```
    avatar: string;
  }
}
```

API Endpoints (Backend)

Fan Journey

POST	/api/requests	# Create new commission request
GET	/api/requests/:id	# Fetch request details
GET	/api/requests/user/:userId	# List user's requests
PATCH	/api/requests/:id	# Update request (save idea, etc.)
POST	/api/director/message	# Send message to AI Director
POST	/api/director/choice	# Select a choice (updates scene)
GET	/api/scene/:id	# Fetch live scene card

Creator Dashboard

GET	/api/creator/requests	# List all creator's requests (with filters)
GET	/api/creator/requests/:id	# Fetch single request
POST	/api/creator/approve/:id	# Approve request
POST	/api/creator/propose/:id	# Propose changes
POST	/api/creator/ask/:id	# Ask a question
POST	/api/creator/decline/:id	# Decline request

Auth

POST	/api/auth/login	# Login
POST	/api/auth/logout	# Logout
POST	/api/auth/register	# Register
GET	/api/auth/me	# Current user

CSS Custom Properties & Animations

Custom Properties

```
:root {
  --color-bg: #FDF8F3;
  --color-panel: #FFFFFF;
  --color-secondary: #F5F0EB;
  --color-text-primary: #302720;
```

```

--color-text-muted: #70625C;
--color-divider: #DED3CB;
--color-rose: #E4A4BD;
--color-deepprose: #84485D;
--color-alert: #E37A6A;
--color-success: #8EB486;

--shadow-subtle: 0 8px 30px rgba(0,0,0,0.04);
--shadow-modal: 0 20px 60px rgba(0,0,0,0.08);
--shadow-light: 0 2px 8px rgba(0,0,0,0.02);

--duration-short: 160ms;
--duration-standard: 200ms;
--duration-long: 400ms;

--timing-ease: cubic-bezier(0.4, 0, 0.2, 1);
--timing-bounce: cubic-bezier(0.175, 0.885, 0.32, 1.275);
}

```

Global Animations

```

@keyframes bounce-in-soft {
  0% { transform: scale(0.9); opacity: 0; }
  50% { transform: scale(1.05); opacity: 1; }
  100% { transform: scale(1); opacity: 1; }
}

@keyframes price-flash {
  0%, 100% { background-color: transparent; }
  50% { background-color: rgba(228, 164, 189, 0.3); }
}

@keyframes slide-up {
  from { transform: translateY(10px); opacity: 0; }
  to { transform: translateY(0); opacity: 1; }
}

.animate-bounce-in {
  animation: bounce-in-soft var(--duration-long) var(--timing-bounce) forwards;
}

.animate-price-flash {
  animation: price-flash 2s ease-out;
}

.animate-slide-up {
  animation: slide-up var(--duration-short) var(--timing-ease) forwards;
}

/* Respect prefers-reduced-motion */
@media (prefers-reduced-motion: reduce) {
  * {

```

```

    animation-duration: 0.01ms !important;
    animation-iteration-count: 1 !important;
    transition-duration: 0.01ms !important;
  }
}

```

Accessibility Checklist

- ☒ **Focus Rings:** 2px espresso outline, 2px offset on all interactive elements
- ☒ **Semantic HTML:** Use `<button>`, `<a>`, `<input>` correctly
- ☒ **ARIA Labels:** Add `aria-label` on icon-only buttons
- ☒ **Color Contrast:** All text meets WCAG AA standards (espresso on cream ✓)
- ☒ **Touch Targets:** 44px minimum on mobile, 48px on desktop
- ☒ **Keyboard Navigation:** All interactions work with Tab + Enter
- ☒ **Reduced Motion:** Respect `prefers-reduced-motion` (disable animations)
- ☒ **Alt Text:** All images have descriptive alt text
- ☒ **Form Accessibility:** Labels linked to inputs via `id` + `htmlFor`
- ☒ **Modals:** Trap focus inside modal, restore on close

Testing Checklist

Unit Tests

- ☐ Button component renders with correct variants
- ☐ RequestCard displays status icon correctly
- ☐ SceneCard calculates budget indicator correctly
- ☐ ProgressTrail highlights active step

Integration Tests

- ☐ Navigate Boutique → AI Director → Review → Confirmation (fan journey)
- ☐ Navigate Creator Dashboard → Detail Modal → Ask Question → Dashboard
- ☐ Filter/sort on Creator Dashboard updates card list
- ☐ Close modal with X button returns to dashboard

E2E Tests (Cypress/Playwright)

- ☐ Fan can select theme and send to creator
- ☐ Creator can approve/propose/ask/decline
- ☐ Pricing updates dynamically as choices change

- ☐ Budget indicator shows red when over budget

Visual Regression Tests

- ☐ All pages match design specs (screenshots)
 - ☐ Responsive breakpoints (mobile, tablet, desktop)
 - ☐ Hover/focus states on all interactive elements
-

Performance Optimization

- **Code Splitting:** Lazy-load modal components
 - **Image Optimization:** Use WebP format, srcset for responsive images
 - **CSS Purging:** Tailwind removes unused styles in production
 - **Memoization:** React.memo for RequestCard list rendering
 - **Debouncing:** Debounce filter/sort inputs
-

Deployment

Frontend (Vercel/Netlify)

```
npm run build
# Deploy dist/ folder
```

Environment Variables

```
VITE_API_BASE_URL=https://api.example.com
VITE_APP_NAME=Fan Director Studio
VITE_DEMO_MODE=true
```

Production Checklist

- ☐ Remove console.logs
 - ☐ Enable gzip compression
 - ☐ Set up error tracking (Sentry)
 - ☐ Configure CORS headers
 - ☐ Enable HTTPS only
 - ☐ Set security headers (CSP, X-Frame-Options, etc.)
-

Future Enhancements

1. **Real AI Director** — Integrate OpenAI API for chat
 2. **Payment Processing** — Stripe/Square integration
 3. **Video Upload** — S3 integration for fan video submissions
 4. **Email Notifications** — SendGrid integration
 5. **Analytics** — Track request flow, conversion rates
 6. **Creator Profiles** — Customizable creator branding
 7. **Mobile App** — React Native version
 8. **Real-time Updates** — WebSocket for live notifications
-

Questions?

Refer to **Design System — Fan Director Studio** pinned note for complete visual specs, color codes, typography, and component HTML snippets.

All pages are linked at:

- Boutique Entrance: <https://p.superdesign.dev/draft/82277dd9-3db3-4c9d-8e8c-272cff83b5fe>
- AI Director: <https://p.superdesign.dev/draft/048b6b8a-1e95-4352-b0eb-32e252b1e30f>
- Review Scene Card: <https://p.superdesign.dev/draft/48723685-c114-4cee-b2c5-6b27b89fc2de>
- Send Confirmation: <https://p.superdesign.dev/draft/a98b81cb-4ff3-43f7-b343-7091c69f78e3>
- Creator Dashboard: <https://p.superdesign.dev/draft/2b65d4ed-45e0-464d-a22b-56a70377752b>
- Creator Detail Modal: <https://p.superdesign.dev/draft/c6e6d108-a7c9-4d51-90b2-4090ebce1a93>
- Ask Question Modal: <https://p.superdesign.dev/draft/5d2c9ba0-0041-4771-b9b3-cc7a17bd7463>
- Decline Confirmation: <https://p.superdesign.dev/draft/7fc2a528-a81a-46a5-b7f4-b81af52a323a>